

Lesson 10

by Lorraine Gaudio

Lesson generated on October 26, 2025



BOISE STATE UNIVERSITY

Contents

1	☒ Ready, Set, R!	3
2	☒ Packages	4
3	Basic Terminology	5
3.1	Types of Sample data	6
4	Choosing the Right Statistical Method	7
5	Descriptive Statistics	8
5.1	Addition	8
5.2	Mode	10
5.3	Mean	10
5.4	Variance	11
5.5	Standard Deviation	11
5.6	Median	12
5.7	Range	12
6	dplyr's summarize()	16
7	dplyr's group_by()	17
8	Explicit vs. Default	18
9	Multiple Statistics	19
10	Multiple Variables	20
10.1	ungroup()	20
11	☒ Practice Space	23
12	☒ Assignment	24
12.1	Task 0	24
12.2	Task 1	24
12.3	Task 2	24
12.4	Task 3	25
12.5	Task 4	25
12.6	Task 5	26
13	Save and Upload	27

14 Today you practiced:

28

1. 🏁 Ready, Set, R!

In lesson 8 and 9, we explored several functions from dplyr (`select()`, `filter()`, `mutate()`, and `case_when()`). This week, we will look at other functions in the dplyr package (`group_by()`, `summarize()` and `count()`) that will speed your ability to compare sub-groups and calculate descriptive statistics.

To begin Lesson 10, follow these steps:

1. Open your course project for RStudio
2. Create a new file. From the file types we have used so far, pick which file type you want to use. (File > New File > ???). Select either R Script, R Markdown, or Quarto Document. What file type you choose to practice the lesson and complete your assignment in is up to you.
3. Type in the code provided in this document as you follow along with the video. Pause the video at any time to answer assignment questions, dig deeper or add memo notes.

Lesson Overview

By the end of Lesson 10 you will be able to:

1. ☐ Remember – List the roles of `group_by()` `count()` and `summarize()`.
2. ☐ Understand – Explain how grouping changes the behavior of summary functions.
3. ☐ Apply – Compute mean, standard deviation, and count for each group in one call.
4. ☐ Analyze – Combine multiple grouping variables.
5. ☐ Evaluate – Decide when to overwrite vs. keep original variables.

Keep these goals in mind as you move through each section.

2. ☒ Packages

Install once (if needed): `install.packages("dplyr"); install.packages("dslabs")`

Load the packages at the start of every session:

```
library(dslabs) # Data science labs package  
library(dplyr)  # Data manipulation package
```

3. Basic Terminology

It occurred to me that we have not yet defined some basic terminology that is important to understand before we dive into descriptive statistics. So, let's take a moment to define some key terms. If these terms are already familiar to you, feel free to skip ahead to the next section.

Population: A population is a collection of objects or individuals that we want to study. It can be finite or infinite. The unknown population number is called a *parameter*.

- **Census:** A census is a complete enumeration of all the elements in a population. It is used to collect data from the entire population.

Sample: A sample is a subset of the population that we actually observe or measure. It is used to make inferences about the population. We use samples because it is often impractical or impossible to collect data from the entire population. The known sample number is called a *statistic*.

- Population Size **N**: The number of elements in the population.
- Sample Size **n**: The number of elements in the sample.

□ Your data set is most likely a sample from a larger population and not a census of the entire population.

1. **Unordered Sample:** A sample where the order of the elements does not matter. For example, if we have a population of 5 elements and we take a sample of 3, the sample {1, 2, 3} is the same as {3, 2, 1}.

Example: 5.9ft, 6.1ft, 5.8ft, 6.0ft, 5.7ft is a sample of heights of 5 individuals from a population of all individuals in a city.

```
heights_sample <- c(5.9, 6.1, 5.8, 6.0, 5.7) # heights in feet
```

The order has no inherent meaning.

2. **Ordered Sample:** A sample where the order of the elements does matter. Time series data, growth measurements, finishing positions in a race, and data sorted from smallest to largest for calculating quantiles or medians are examples of ordered samples.

Example: If we take 5 samples of the height of one individual by age, we have an ordered sample.

```
growth_data <- c(42, 45, 48, 51, 54) # Heights in inches by age
race_positions <- c("Kim", "Alex", "Jordan", "Taylor") # Order indicates ranking
sorted_temps <- c(72.1, 73.4, 75.2, 76.8, 79.3) # Ordered for statistical analysis
```

Ordered sample is about whether the sequence of selection matters in your sampling process. The data can simply be ordered by size.

Ordinal data is about whether your data values have a meaningful natural ranking order, the intervals between categories aren't necessarily equal. Likert scale or types of other hierarchical values like academic hierarchies.

3.1 Types of Sample data

Categorical Data: Data that can be divided into categories or groups. For example, the colors of cars in a parking lot (red, blue, green, etc.).

Numerical Data: Data that can be measured or counted. For example, the heights of individuals in a population (in feet or meters).

- **Discrete Data:** Numerical data that can take on only specific values. For example, the number of students in a class (1, 2, 3, etc.). Cannot take on fractional values.
- **Continuous Data:** Numerical data that can take on any value within a range. For example, the height of individuals in a population (5.0ft, 5.1ft, 5.2ft, etc.).

□ Challenge: What type of data is `growth_data`, `race_positions`, and `sort_temps`? Explain your answer in a memo to demonstrate learning skills.

4. Choosing the Right Statistical Method

When analyzing data, it is important to choose the appropriate statistical method based on the type of sample data you have. Here are the steps to follow:

1st Step: Check the type of sample data

2nd Step: Apply the appropriate statistical method

5. Descriptive Statistics

Descriptive statistics are used to summarize and describe the main features of a dataset. They provide a way to understand the distribution, central tendency, and variability of the data.

5.1 Addition

Length, sum, count are all ways to add up values in a dataset. What is the difference between them?

5.1.1 Length

```
# Count number of values in mpg column  
length(mtcars$mpg)
```

```
## [1] 32
```

Length counts the number of values in the set of numbers. In other words, it tells us how many observations we have. In a data table, it tells us how many rows are in a column.

5.1.2 Sum

```
# Sum step 1: Sum the values  
sum_sample <- sum(mtcars$mpg)
```

In statistics, summation is represented by the Greek letter sigma (Σ). It indicates that we are summing a series of values.

$$\sum_{i=1}^n$$

When you see $\sum_{i=2}^5 a$, this means:

- Start with $i = 2$ (the second value in the set of numbers)
- Add the value a to your sum

- Increase i by 1
- Repeat until you reach $i = 5$ (the fifth value in the set of numbers)
- Then stop

This is adding a four times (once each for $i = 2, i = 3, i = 4$, and $i = 5$).

$$\sum_{i=2}^5 a \Rightarrow a_2 + a_3 + a_4 + a_5 = 4a = (5 - 2 + 1)a$$

table() function to create a frequency table

```
table(mtcars$cyl)
```

```
##
```

```
## 4 6 8
```

```
## 11 7 14
```

□ Reflect: Why did we use the `table()` function instead of `sum()` to count the number of occurrences of each unique value in the `cyl` column? Why did we `sum()` the values in the `mpg` column and not use `table`? In other words, would it make sense to `sum(mtcars$cyl)`? Create a memo to demonstrate your learning skills.

5.1.3 Count

The **dplyr** function `count()` is a convenient way to count the number of occurrences of each unique value in a categorical variable.

Use `?count` to learn more about the function.

Count occurrences of each unique value in the cyl column

```
cyl_counts <- mtcars %>%
```

```
  count(cyl)
```

```
cyl_counts
```

cyl	n
4	11
6	7
8	14

`n` is the number of times that each unique value appears in the `cyl` column.

□ Comment: What is the difference between `length()`, `sum()` and `count()`? Explain your answer in a memo to demonstrate learning skills.

5.2 Mode

The mode is the most frequent value(s) in a dataset. Mode is usually used for categorical data or discrete data where we want to find the most common category or value. It is not commonly used for continuous data.

```
# Create a frequency table  
table(mtcars$cyl)
```

```
##  
## 4 6 8  
## 11 7 14
```

☐ Comment (example): 8 cylinders occur most frequently (14 times) in the cyl column of the mtcars dataset.

Note: If no value occurs more than once, then all are the mode or no mode. You get to pick.

☐ Metacognition: Which function table() or count() do you prefer for finding the mode? Explain your answer in a memo to demonstrate learning skills.

5.3 Mean

The mean is the average of a set of numbers. It is calculated by summing all the values and dividing by the number of values.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Note: We use sum_sample from above.

```
# Mean step 2: Divide by the number of values  
mean_sample <- sum_sample / length(mtcars$mpg)  
mean_sample
```

```
## [1] 20.09062
```

```
# OR use mean() function directly  
mean_value <- mean(mtcars$mpg)  
mean_value
```

```
## [1] 20.09062
```

5.4 Variance

The variance is a measure of how spread out the values in a dataset are. It is calculated as the average of the squared differences from the mean.

Variance is calculated by first finding the difference between each value and the mean, squaring those differences, summing them up, and then dividing by the number of values minus one (for a sample variance).

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{(n - 1)}$$

```
# Variance step 3: Calculate the variance
```

```
variance <- sum((mtcars$mpg - mean_value)^2) / (length(mtcars$mpg) - 1)
variance
```

```
## [1] 36.3241
```

```
# OR use var() function directly
```

```
variance <- var(mtcars$mpg)
variance
```

```
## [1] 36.3241
```

5.5 Standard Deviation

The standard deviation is a measure of how spread out the values in a dataset are. It is calculated as the square root of the variance.

$$s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

```
# Standard Deviation step 4: Calculate the standard deviation
```

```
std_dev <- sqrt(variance)
std_dev
```

```
## [1] 6.026948
```

```
# OR use sd() function directly
```

```
std_dev <- sd(mtcars$mpg)
std_dev
```

```
## [1] 6.026948
```

5.6 Median

The median is the middle value of a set of numbers when they are arranged in order. If there is an even number of values, the median is the average of the two middle values.

Step 1: Sort the data in ascending order.

Step 2: Check the size of the sample (n):

- If n is odd, the median is the middle value:

$$\text{median} = x_{(\frac{n+1}{2})}$$

- If n is even, the median is the average of the two middle values:

$$\text{median} = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$$

Where $x_{(i)}$ represents the i th ordered value in the sample.

```
# Median step 1: Sort the data
sorted_data <- sort(mtcars$mpg)
sorted_data
```

```
## [1] 10.4 10.4 13.3 14.3 14.7 15.0 15.2 15.2 15.5 15.8 16.4 17.3 17.8 18.1 18.7
## [16] 19.2 19.2 19.7 21.0 21.0 21.4 21.4 21.5 22.8 22.8 24.4 26.0 27.3 30.4 30.4
## [31] 32.4 33.9
```

```
# Median step 2: Get the size of the sample
length(sorted_data) # Get the size of the sample
```

```
## [1] 32
```

```
# Or use the median() function
median(mtcars$mpg)
```

```
## [1] 19.2
```

5.7 Range

This simplest measure of variability is the range, which is the difference between the largest and smallest values in a dataset. $x(n) - x(1)$

Sensitive to outliers, or extreme values, which can skew the range.

5.7.1 Interquartile Range (IQR)

The interquartile range (IQR), or the fourth spread, is the range of the middle 50% of the data. It is calculated as:

$$IQR = Q_3 - Q_1$$

IQR is less sensitive to outliers than the range because it focuses on the central portion of the data.

Where Q_1 is the first quartile (25th percentile) and Q_3 is the third quartile (75th percentile).

Remove lower and upper 25% removes extreme values and determine the range between the first and third quartiles.

5.7.2 Percentiles

A percentile is a value below which a given percentage of observations fall. The $100 \times p$ -th percentile (where p is between 0 and 1) is the value such that $p \times 100\%$ of the data falls below that value.

100% of the data are less than or equal to the 100% percentile. Divide the data into 100 equal parts.

Quartile: divide the whole data range into four equal parts.

Q_1 is the lower quartile (25th percentile), Q_2 is the median (50th percentile), and Q_3 is the upper quartile (75th percentile).

5.7.2.1 Finding Percentiles in Ordered Data

1st: Sort the data from the smallest to the largest

```
print(mtcars$mpg)
```

```
## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

```
# Sort the mpg column in ascending order
sorted_mpg <- sort(mtcars$mpg)
```

```
print(sorted_mpg)
```

```
## [1] 10.4 10.4 13.3 14.3 14.7 15.0 15.2 15.2 15.5 15.8 16.4 17.3 17.8 18.1 18.7
## [16] 19.2 19.2 19.7 21.0 21.0 21.4 21.4 21.5 22.8 22.8 24.4 26.0 27.3 30.4 30.4
## [31] 32.4 33.9
```

2nd: Find the position value

```
# Find minimum value of mpg in mtcars
min_mpg <- sorted_mpg[1]
min_mpg
```

```
## [1] 10.4
```

```
# Or simply use min() function
min_mpg <- min(mtcars$mpg)
min_mpg
```

```
## [1] 10.4
```

□ Comment: How is the minimum value found using [1] in the sorted data? Explain your answer in a memo to demonstrate learning skills.

Type ?ceiling to learn about the ceiling() function.

```
# Find the 25th percentile (Q1) of mpg in mtcars
n <- length(mtcars$mpg)
Q1_position <- 0.25 * (n + 1)
Q1_value <- sorted_mpg[ceiling(Q1_position)]
Q1_value
```

```
## [1] 15.5
```

Type ?quantile to learn about the quantile() function.

```
# Or simply use quantile() function
Q1_value <- quantile(mtcars$mpg, 0.25)
Q1_value
```

```
## 25%
## 15.425
```

```
# Find the median (50th percentile) of mpg in mtcars
median_position <- 0.5 * (n + 1)
median_value <- sorted_mpg[ceiling(median_position)]
median_value
```

```
## [1] 19.2
```

```
# Or simply use median() function
median_value <- median(mtcars$mpg)
median_value
```

```
## [1] 19.2
```

```
# Find the 75th percentile (Q3) of mpg in mtcars
Q3_position <- 0.75 * (n + 1)
Q3_value <- sorted_mpg[ceiling(Q3_position)]
Q3_value
```

```
## [1] 22.8
```

```
# Or simply use quantile() function
Q3_value <- quantile(mtcars$mpg, 0.75)
Q3_value
```

```
## 75%
## 22.8
```

```
# Find maximum value of mpg in mtcars
max_mpg <- sorted_mpg[n]
max_mpg
```

```
## [1] 33.9
```

```
# Or simply use max() function
max_mpg <- max(mtcars$mpg)
max_mpg
```

```
## [1] 33.9
```

5.7.3 Review summary()

You can use the `summary()` function to get a quick overview of the distribution of a numeric variable, including the minimum, 1st quartile, median, mean, 3rd quartile, and maximum values.

```
?summary
data("mtcars")
summary(mtcars) # Summarizes all columns at once
summary(mtcars$mpg) # Summarizes just the mpg column
```

□ And now onto the main event: `summarize()` and `group_by()`

6. dplyr's summarize()

You can use `summarize()` to compute summary statistics like the mean, median, etc.

?summarize

- The SYNTAX data `%>% summarize(new_column_name = some_operation(column_name))`
- The GOAL: Find the overall mean mpg for all cars.

```
# □ Test:  
mtcars %>%  
  summarize(mean_mpg = mean(mpg))
```

- Comment (example): The code above creates a new column called `mean_mpg` that contains the mean of the `mpg` column from the `mtcars` dataset.
- The GOAL: Find several summary statistics for `mpg`: mean, max, and count of cars.

```
# □ Test:  
mtcars %>%  
  summarize(mean_mpg = mean(mpg),  
            max_mpg = max(mpg),  
            count = n())
```

- NOTICE: In the code chunk above, why is there not piping `%>%` after the calculation of `mean()`, `max()` and `n()`? Explain your answer in a memo to demonstrate learning skills.
- Reflect: □ Compare `summary()` with `summarize()` Explain your answer and how one or the other might be useful in different situations. Compose a short memo to demonstrate learning skills.

7. dplyr's group_by()

group_by() changes the behavior of summary functions.

- The SYNTAX

```
data %>% group_by(variable1, variable2, ...) %>% some_operation()
```

- The GOAL: Find the mean mpg for each cylinder count.

```
# □ Test:  
cyl_means <- mtcars %>%  
  group_by(cyl) %>%  
  summarize(mean_mpg = mean(mpg))
```

- Comment (example): group_by(cyl) treats each unique value separately, and summarize() runs mean(mpg) **per group**. mean_mpg is the *explicit* name of the new column.

```
# □ Verify  
cyl_means
```

- Metacognition: Reflect on why is it useful or why might it make sense to group cars by cylinders before summarizing mpg. Then determine how might you apply this technique to your signature assignment dataset? Type a memo to demonstrate learning skills.

8. Explicit vs. Default

□ Question (example): What is the *default* name of the summary column when you do not name it explicitly?

```
# □ Test:  
mtcars %>%  
  group_by(cyl) %>%  
  summarize(mean(mpg))
```

□ Comment (example): The default name is the code itself. Default names can be confusing when sharing results. Naming columns makes tables clearer. Use it whenever you share results.

□ NOTICE: that we chain `group_by()` and `summarize()` together using the pipe operator `%>%`. This allows us to first group the data by the specified variable(s) and then apply the summary function to each group.

9. Multiple Statistics

- The GOAL: We can create several summary columns in one call, separated by commas.

```
# □ Test:
cyl_stats <- mtcars %>%
  group_by(cyl) %>%
  summarize(mean_mpg = mean(mpg),
            sd_mpg   = sd(mpg),
            n        = n())
# □ Verify:
cyl_stats
```

?n to learn about the n() function.

- Comment (example): n() counts rows in each group. This is similar to the function length() which counts the number of values in a vector or row in a data set. It is different from length because n() works within summarize() and counts the number of rows in each group created by group_by(). In this way, n() is like count(), but it is used within summarize(). Using n() is a quick way to see how many observations are in each group.

We *name* each new column using new_name = calculation.

10. Multiple Variables

`group_by` one column creates one group for each unique value. `group_by` two columns creates one group for each unique combination.

□ Question (example): How many groups will `group_by(cyl, am)` create?

```
# Recall that mtcars$cyl has three unique values:  
unique(mtcars$cyl)  
# Recall that mtcars$am has two unique values:  
unique(mtcars$am)
```

□ Prediction (example): Two variables with 3 and 2 unique values creates $3 \times 2 = 6$ groups.

```
# □ Test:  
mtcars %>%  
  group_by(cyl, am) %>%  
  summarize(avg_mpg = mean(mpg),  
            count = n())
```

□ Comment (example): - Group 1: `cyl = 4, am = 0` (4-cylinder automatic cars)

- Group 2: `cyl = 4, am = 1` (4-cylinder manual cars)
- Group 3: `cyl = 6, am = 0` (6-cylinder automatic cars)
- Group 4: `cyl = 6, am = 1` (6-cylinder manual cars)
- Group 5: `cyl = 8, am = 0` (8-cylinder automatic cars)
- Group 6: `cyl = 8, am = 1` (8-cylinder manual cars)

10.1 ungroup()

With two grouping vars, the default keeps the earlier group(s). With `.groups = "drop"` (or `ungroup()`), the result is not grouped.

A) Default behavior: keeps earlier groups

□ Goal: Show that with two groups (`cyl, gear`), `summarize()` keeps the earlier group (`cyl`) by default.

```
# □ Test:
# A) Default behavior: "drop_last" (keeps earlier groups)
cyl_gear_stats <- mtcars %>%
  group_by(cyl, gear) %>%
  summarize(mean_mpg = mean(mpg)) # no .groups set
```

```
# □ Verify:
group_vars(cyl_gear_stats) # what groups remain?
```

```
# □ Verify:
is_grouped_df(cyl_gear_stats) # expect TRUE
```

□ Comment (example): The Default = drop_last from (cyl,gear) down to (cyl).

You'll want to be explicit when you don't want any grouping left.

B) Make it explicit inside summarize()

□ Goal: Return a plain, ungrouped table by telling summarize() to drop all groups.

```
# □ Test:
# B) Make it explicit: drop *all* groups in the result
cyl_gear_stats_drop <- mtcars %>%
  group_by(cyl, gear) %>%
  summarize(mean_mpg = mean(mpg), .groups = "drop")
```

```
# □ Verify:
group_vars(cyl_gear_stats_drop) # expect character(0)
```

```
# □ Verify:
is_grouped_df(cyl_gear_stats_drop) # expect FALSE
```

□ Comment (example): Using .groups = "drop" tells summarize() to remove all grouping after the summary operation, resulting in a plain data frame without any grouping structure.

C) Alternative: drop groups after summarizing

Goal: Achieve the same ungrouped result using ungroup() after summarize()

```
# □ Test:
# C) Alternative: drop groups after summarizing
cyl_gear_stats_ungroup <- mtcars %>%
  group_by(cyl, gear) %>%
  summarize(mean_mpg = mean(mpg)) %>%
  ungroup()
```

```
# □ Verify:  
group_vars(cyl_gear_stats_ungroup) # expect character(0)
```

```
# □ Verify:  
is_grouped_df(cyl_gear_stats_ungroup) # expect FALSE
```

□ Comment (example): Using `ungroup()` after `summarize()` removes all grouping from the resulting data frame, similar to using `.groups = "drop"` within `summarize()`. Both methods achieve the same outcome of returning an ungrouped data frame.

Pick either B) or C) based on your coding style preference.

11. ☒ Practice Space

Below are optional activities you can do to practice the skills in this lesson. Pick one or more to complete to earn learning skill points!

☐ Reflect: How many rows would you expect if each state has 6 diseases? Make a prediction and test it. Then write a memo ☐ Commenting about the results or ☐ Explaining why this will be useful.

☐ Practice: Pipeline. Fill in the Blanks

For `mtcars`, compute minimum and maximum horsepower for each transmission type (0 = automatic, 1 = manual). Store as **`transmission_hp`**.

```
transmission_hp <- mtcars %>%  
  ____ (____) %>%  
  ____ (min_hp = min(____),  
        max_hp = max(____))
```

`transmission_hp`

☐ Explore and Play: Try changing horse power to miles per gallon or weight and rerun. Create a memo note, ☐ Commenting about the results or ☐ Explaining why this will be useful.

☐ Practice: Replace the blanks (____) and run.

You'll use the `heights` dataset and use the functions `group_by`, `summarize`, `mean`, `sd`, `n`

```
data("heights")  
?heights
```

```
heights %>%  
  ____ (sex) %>%  
  ____ (mean_height = ____ (height),  
        sd_height = ____ (height),  
        n = ____ ()) -> height_descriptives
```


12. ☒ Assignment

Replace each _____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment. When finished, save **BOTH** code script and your .RData workspace and upload.

Where appropriate, record: ☐ / ☐ Goal → ☐ Code → ☐ Checks → ☐ / ☐ Reflection (≤ 50 chars).

12.1 Task 0

☐ Library it up!

Make sure there is script in your document to load dplyr and dslabs packages so your functions / datasets load.

12.2 Task 1

Single grouping

☐ Using stars, compute mean temperature **AND** mean magnitude for each type. Name the result **twinkle_twinkle**.

☐ Restate the Goal

```
#☐ Code  
_____- <- _____
```

```
#☐ Checks
```

☐ Did it work? What happened?

12.3 Task 2

Single grouping + three stats

☐ Pick ONE numeric variable in olive (not region, area), group by region, return mean, sd, and n. Store as **Olive_Garden**.

☐ Restate the Goal

```
# Code
_____ <- _____
```

```
# Checks
```

In a short comment, note the region with the highest mean value.

☐ Comment: “_____”

12.4 Task 3

Multiple grouping

☐ From `us_contagious_diseases`, `group_by` state AND disease then summarize the mean count. Keep only the three diseases with the highest national mean counts. Store as **Infectious_Burden**.

☐ Hint: **National means by disease**. Group the dataset by disease and summarize the mean of count across all rows. Pick the top 3 diseases. Arrange the diseases by their national mean and keep the top three.

```
data("us_contagious_diseases")
# National means per disease
_____ <- us_contagious_diseases %>%
  _____(_____) %>%
  _____(national_mean = _____(_____), _____ = _____) %>%
  _____(desc(_____), _____) %>%
  head(n = _____)
```

```
# Checks
```

☐ Did it work? What happened?

12.5 Task 4

Putting it all together

☐ Using dplyr's `starwars` data, create a table of **per-group descriptive statistics** by **two grouping variables**. Use several statistics and more than one numeric variable in a single `summarize()` call. Return an **ungrouped** result. Name it `starwars_summary`.

1. Inspect: Open the help page and glimpse the data. Identify **two categorical** columns you can group by (e.g., species and sex) and **two numeric** columns to summarize (e.g., height and mass).

```
data("starwars")
```

2. In one code chunk:

- Use `filter()` to remove rows with missing values so your groups are well-defined.
- `group_by()` your two categorical variables.
- `summarize()` to compute *at least* **two descriptive statistics** (e.g., mean, sd, n) for **each numeric variable**.
- Drop groups to return an ungrouped result.
- Name the result **starwars_summary**.

```
# Code  
_____-<-_____
```

```
# Checks
```

☐ Did it work? What happened?

12.6 Task 5

☐ Reflection

☐ Write a short paragraph reflecting why we need to do `group_by()` come BEFORE `summarize()`?

☐ EXPLANATION: “___”

13. Save and Upload

1. You will be submitting **both** the script document of your choice and the workspace file. The workspace file saves all the objects in your environment that you created in this lesson. You can save the workspace by running the following command:

```
save.image("Assignment10_Workspace.RData")
```

Or you can click the “Save Workspace” button in the Environment pane.

☐ **Always save the document before closing.**

2. Find the assignment in this week’s module in Canvas and upload **both** the document that contains your code and memos (.R, .Rmd, or .qmd) and the workspace file (.RData).

14. Today you practiced:

- Compared overall vs. grouped summaries.
- Used `group_by()` `count()` and `summarize()` to calculate descriptive statistics.
- Named summary columns for clarity.
- Counted group sizes with `n()`.
- Grouped by multiple variables for richer insights.

☐ Practice writing one-line summaries on your own data and the syntax will quickly become second nature.